

VersaAccel: A Versatile Configurable Accelerator for Diverse Sparse-Dense Matrix Operators

Minjin Tang^{ID} and Mei Wen^{ID}

Abstract—Matrix operators are fundamental to various applications, particularly in deep learning. While early models relied on dense operations, techniques like pruning have introduced sparsity, leading to a mix of dense and sparse operator types. Most existing accelerators are specialized for specific operators and perform poorly in mixed scenarios, while those supporting multiple operators often lack flexibility and suffer from suboptimal performance. To overcome these limitations, we propose VersaAccel, a configurable accelerator for sparse and dense matrix operators. It supports four distinct configurations, each optimized for a set of operators. A key feature of our design is its adaptive configuration selection mechanism, driven by a lightweight cost model that explicitly evaluates the performance-energy tradeoff between available options. This allows VersaAccel to dynamically choose the most efficient configuration—opting for higher performance when the gain outweighs the energy cost, or prioritizing energy efficiency when appropriate. Experimental results demonstrate that VersaAccel achieves an average performance/area improvement of $3.10\times$ across multiple operators [matrix-vector multiplication (MV), matrix multiplication (MM), sparse MV (SpMV), sparse MM (SpMM), and sparse matrix-SpMM (SpMSpM)] and $4.06\times$ on full model evaluations (ResNet18, VGG16, LLaMA2-7B, and BERT-Base), compared to mainstream accelerators.

Index Terms—Configurable hardware, sparse/dense operators.

I. INTRODUCTION

WITH the advancement of deep learning and high-performance computing, matrix computations have become indispensable in fields such as scientific computing [1], [2], graph analysis [3], [4], and recommendation systems [5], [6]. Matrix multiplication (MM) and matrix-vector multiplication (MV) represent the primary computational burdens in deep learning models, particularly in architectures like transformers.

Current hardware accelerators, such as NVIDIA GPUs and Google TPUs [7], have been extensively optimized for dense matrices; however, they falter when it comes to handling sparse

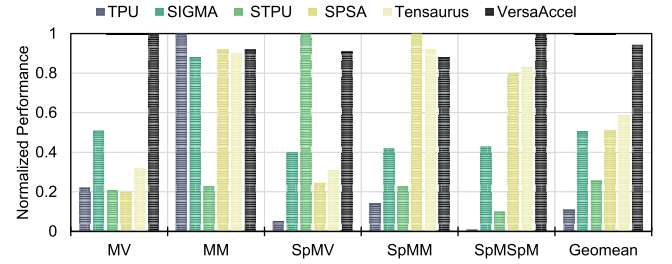


Fig. 1. Comparison of the performance of different accelerators when calculating different operators.

matrices, primarily due to the irregular data access patterns and low computational density associated with sparse operators. Although certain specialized architectures, such as SIGMA [8], STPU [9], and SPSA [10], have enhanced the performance of sparse operators, they often lack support for a diverse range of operators.

Moreover, attempts to develop architectures that accommodate multiple operators, such as Trapezoid [11] and Tensaurus [12], face challenges related to insufficient operator coverage and performance tradeoffs. Thus, the design of flexible and efficient hardware architectures to meet the demands of mixed operator computations is of paramount importance.

To further illustrate the problem, we conducted evaluations of various accelerators on multiple operators. As shown in Fig. 1, when performing computations on MV, MM, sparse MV (SpMV), sparse MM (SpMM), and sparse matrix-SpMM (SpMSpM), existing accelerators specialized for specific operators, such as TPU [7], SIGMA [8], STPU [9], and SPSA [10], fail to handle the other operators efficiently. Although Tensaurus [12] can support multiple operators, it still performs poorly on certain operators (MV/SpMV). In contrast, our proposed VersaAccel demonstrates good performance across all operators and ultimately achieves the highest overall performance.

We have summarized the primary issues and challenges that current hardware accelerators encounter when supporting a variety of operators.

1) *Performance Optimization Contradiction*: Dense and sparse operators exhibit significant differences in data access patterns and computational requirements, and optimizing for one type often sacrifices performance for the other. For example, TPUs perform excellently in dense computations but experience performance drops in sparse computations.

2) *Low Resource Reusability*: Hardware designs optimized for specific operators (such as dedicated sparse accelerators)

Received 20 February 2025; revised 15 August 2025 and 23 November 2025; accepted 25 November 2025. Date of publication 2 December 2025; date of current version 22 June 2026. This work was supported by the Natural Science Foundation of Hunan Province under Grant 2024JJ6470. This article was recommended by Associate Editor X. Zhang. (Corresponding author: Mei Wen.)

Minjin Tang is with the Defense Innovation Institute, Academy of Military Science, Beijing 100073, China, and also with the Key Laboratory of Advanced Microprocessor Chips and Systems, National University of Defense Technology, Changsha 410073, China.

Mei Wen is with the Key Laboratory of Advanced Microprocessor Chips and Systems, National University of Defense Technology, Changsha 410073, China (e-mail: meiwen@nudt.edu.cn).

Digital Object Identifier 10.1109/TCAD.2025.3639392

have low resource reusability, making it difficult to adapt to diverse task loads, resulting in inefficiencies during mixed computations.

3) *Lack of Unified Architecture*: Current hardware typically adopts fixed data flows and computation paths, making it challenging to efficiently support various operators, leading to underutilization of resources in both dense and sparse computations.

Tackling those challenges, there is a need for a hardware architecture design that is flexible, efficient, and scalable, capable of adapting to the computational demands of different operators while balancing performance and energy efficiency.

To meet this need, this article proposes a novel modular hardware architecture that flexibly supports a variety of dense and sparse operators. The architecture is centered around three core modules: the distribution network, the multiplication unit network, and the reduction network. A configurable mechanism determines the activation and connection of these modules, forming various flexible subconfigurations. This design balances the performance of dense and sparse operators, significantly enhances resource utilization and energy efficiency, and offers good scalability to accommodate future emerging computational demands.

The main contributions and innovations of this article are as follows.

- 1) We propose a modular hardware architecture that flexibly adjusts the connections of the distribution network, multiplication unit network, and reduction network through parameterized configurations, supporting various dense and sparse operators, thereby enhancing resource reuse and energy efficiency.
- 2) We design a cost model to optimize the configuration selection of custom accelerators under different operators, explicitly considering the impact of bandwidth, which enables efficient resource utilization.
- 3) Our design provides a novel solution for hardware architectures that support multiple operators and offers important references for future hardware designs in sparse computing and emerging computational paradigms.

Experimental results show that the proposed architecture achieves significant performance improvements for both dense and sparse operators compared to existing mainstream hardware accelerators, while also demonstrating excellent area efficiency.

II. BACKGROUND AND MOTIVATION

A. Matrix Operators

Matrix operators are central to deep learning, with dense and sparse matrix operators playing critical roles in different contexts. Dense operators, characterized by high computational density and regular data access patterns, are widely utilized in various applications, such as the self-attention mechanism in transformers, convolution acceleration in CNNs (e.g., through techniques like im2col), and MV in fully connected layers. On the other hand, sparse operators focus on handling sparse data structures, which is essential in scenarios like the multiplication of sparse adjacency matrices with feature matrices in

GNNs and long-sequence processing in sparse attention mechanisms. Notably, even within a single application, dense and sparse operators often coexist. For instance, in GNNs, sparse operators are used for aggregating information from adjacency matrices, while dense operators are employed for feature transformations. Similarly, in the sparse attention mechanism of transformers, sparse matrix optimizations are applied to the attention patterns while retaining dense operators for processing other components. This interplay allows for more efficient computation and better performance across various deep learning models.

B. Accelerator for Specific Operators

Current hardware accelerators encounter significant performance bottlenecks when supporting these sparse and dense operators. Mainstream hardware, such as GPUs and TPUs, has undergone extensive optimization for dense operators through tensor cores and pipelined parallelism. However, when dealing with sparse operators, the irregularity of data results in idle computational units, leading to a marked decline in resource utilization. On the other hand, specialized sparse accelerators, like SCNN [13] and ExTensor [14], enhance the performance of sparse operators by optimizing sparse data formats and dynamic scheduling, yet they often lack the capability to support dense operators, resulting in lower resource reuse rates. Furthermore, as deep learning models grow increasingly complex, the diversity of operator types expands, and the occurrence of hybrid computation scenarios involving both dense and sparse operators rises, rendering existing hardware inadequate in terms of flexibility and energy efficiency.

C. Accelerator Architecture Features

From the perspective of accelerator architecture design, the core components can be categorized into distribution networks, computation networks, and reduction networks. In the design of dense operator accelerators, these three networks are often implicitly interconnected; for instance, computation units simultaneously perform forwarding and reduction functions. In architectures such as TPUs and pulsed arrays, adjacent computation units can exchange input and output data, thereby maximizing the reusability of dense computations while minimizing the overhead associated with data distribution networks. To support sparse operators on such tightly coupled arrays, many designs [9], [10], [15], [16] preprocess sparse data by rearranging and packing dense data to reduce the proportion of zero values, transforming it into “denser” data for computation, while attempting to minimize modifications to the hardware. However, this preprocessing has its limitations and cannot entirely eliminate zero-value elements. In addition, some sparse accelerators [11], [17], [18], [19] utilize hardware-level intersection operations to process only nonzero elements. To facilitate flexible data distribution, the three-layer networks typically employ explicit connections, such as crossbars and Benes networks. Concurrently, to manage the reduction of partial sums, enhanced addition trees like FAN [8] and MRN [17] are utilized. Ultimately, this leads to an architecture characterized by a layered explicit

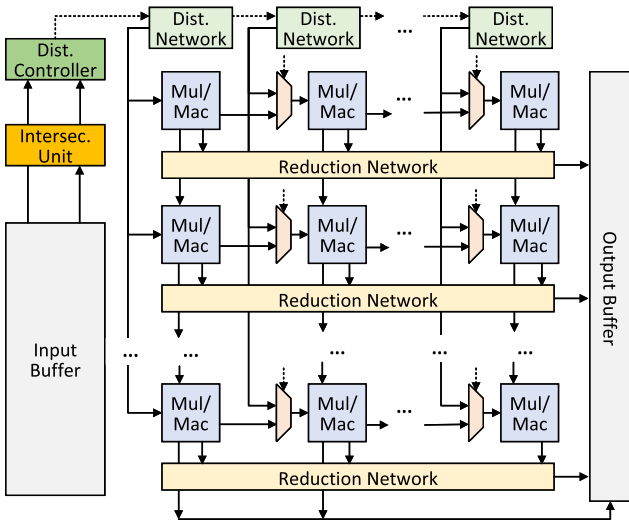


Fig. 2. Overview of VersaAccel architecture.

connection structure that integrates distribution, computation, and reduction.

D. Motivations

1) *Necessity of Multioperator Support*: As deep learning models continue to diversify and grow in complexity, the alternating computations between various operator types, particularly between dense and sparse operators, have become increasingly prevalent. Consequently, accelerators must possess the capability to support a multitude of operators. However, due to the differing computational characteristics of these operators, achieving acceleration for all types using a singular architecture proves to be a formidable challenge. Existing hardware architectures tend to favor larger arrays of computation units (such as MM units) when supporting dense operators, while sparse operators rely heavily on efficient indexing and the handling of sparse data structures, leading to conflicting hardware requirements. This presents a significant challenge for hardware design.

2) *Balancing Performance and Energy Efficiency*: Generic hardware designs can facilitate the acceleration of multiple operators, yet they often struggle to achieve a balance between performance and energy efficiency. To attain this equilibrium within a unified architecture, a thorough analysis of the characteristics of different operators is essential, alongside the introduction of flexibility into the hardware design. A critical issue lies in the dynamic allocation of computational resources to meet the diverse demands of various operators. Moreover, achieving a harmonious coexistence of both performance and energy efficiency through joint optimization tailored to the characteristics of dense and sparse operators remains a challenge.

III. VERSAACCEL ARCHITECTURE

As illustrated in Fig. 2, the overall architecture of VersaAccel is depicted, with its main modules being the distribution network (D), multiplication network (M), and reduction network (R). The intersection module serves as the sparse

functional unit, and the architecture is also equipped with input and output buffers. The processing elements (PEs) are organized in a 2-D array to form the multiplication network. The distribution network is adapted from the Benes network mentioned in SIGMA [8], while the reduction network is modified from the merge-reduction network (MRN) used in Flexagon [17]. In addition, we employ a grouped subnetwork approach for deployment, which reduces the depth of the network while enhancing parallelism and scalability. Specifically, each PE is capable of performing single-precision multiplication or multiply-accumulate operations. For PEs aligned in the same column, a shared distribution network module is provided, while PEs in the same row share a common reduction network module. The distribution controller is responsible for managing data retrieval in the distribution network. It can directly fetch data from the input buffer or, based on the output from the intersection unit, selectively retrieve data from the input buffer. This dual-mode operation enhances flexibility and efficiency, allowing the controller to adapt to both dense and sparse data patterns. By leveraging the intersection unit's output, the distribution controller ensures that only relevant data is processed, optimizing resource utilization and reducing unnecessary data movement. In addition, the connections between adjacent PEs are preserved, similar to a systolic array. This architectural design allows us to drive different modules and adjust the explicit and implicit relationships between the distribution network, multiplication network, and reduction network, thereby enabling various configurations (detailed in Sections III-A–III-D).

Based on the connection relationships among the three core components (D, M, and R), the architecture is divided into four basic subconfigurations. The classification depends on whether the connections between the two layers of networks are explicit or implicit. Table I summarizes the characteristics, advantages, and disadvantages of the four configurations (DMR, D-MR, DM-R, and D-M-R) in terms of their architectural design and performance tradeoffs. Each configuration offers unique benefits and limitations, making them suitable for different computational scenarios and optimization goals.

It is important to note that all configurations are capable of supporting different operators through appropriate mapping strategies, although their performance may vary significantly. To illustrate this, we will use a 2×2 -scale VersaAccel to demonstrate how different operators are mapped under four configurations. As shown in Fig. 3, we select five representative operators for comparative analysis, where the input matrices A (left operand) and B (right operand), as well as the output result C, are all represented in coordinate format.

Specifically, Fig. 3(a) illustrates a 4×4 dense matrix multiplying with a 4×1 vector, where the superscripts in the output vector clearly indicate the partial sum indices constituting the final results. Fig. 3(b) demonstrates the multiplication process between two dense matrices, where a 4×4 matrix multiplies with a 4×2 matrix to produce a 4×2 result matrix. When sparse data is introduced, as shown in Fig. 3(c), the product of a 4×2 sparse matrix and a vector still yields a dense vector, but with altered partial sum composition patterns. Fig. 3(d) and (e) further exhibits operations involving

TABLE I

FEATURES, ADVANTAGES, AND DISADVANTAGES OF DIFFERENT CONFIGURATION TYPES IN VERSAACCEL. SUPPORT FOR MULTIPLE OPERATORS CAN BE ACHIEVED THROUGH SUPPORT FOR MULTIPLE CONFIGURATIONS

Type	Feature	Description	Advantages	Disadvantages
DMR	Implicit-Implicit	The three levels are merged into one level, where the PE unit simultaneously handles data transfer, multiplication, and addition.	Simple structure, high area efficiency, and high computational density.	Lack of flexibility.
D-MR	Explicit-Implicit	The MR levels are merged into one level. The PE unit performs multiplication and addition, and only partial sums between connected PEs in a systolic manner.	The MR structure has similar latency for multiplication and addition compared to the M-R structure but eliminates the overhead of the addition tree for reduction.	Multiplication and addition are tightly coupled, leading to poor support for sparse operations.
DM-R	Implicit-Explicit	The DM levels are merged into one level, and the addition network is responsible for partial sum reduction.	DM leverages PEs to handle partial data transfer, reducing communication overhead and alleviating bandwidth pressure compared to D-M.	Input data is tightly coupled with multiplication units, increasing dependency.
D-M-R	Explicit-Explicit	Each level is separated, with data distribution, multiplication, and reduction performed in distinct stages.	High flexibility and excellent support for sparse operations.	Increased area overhead and more complex control logic.

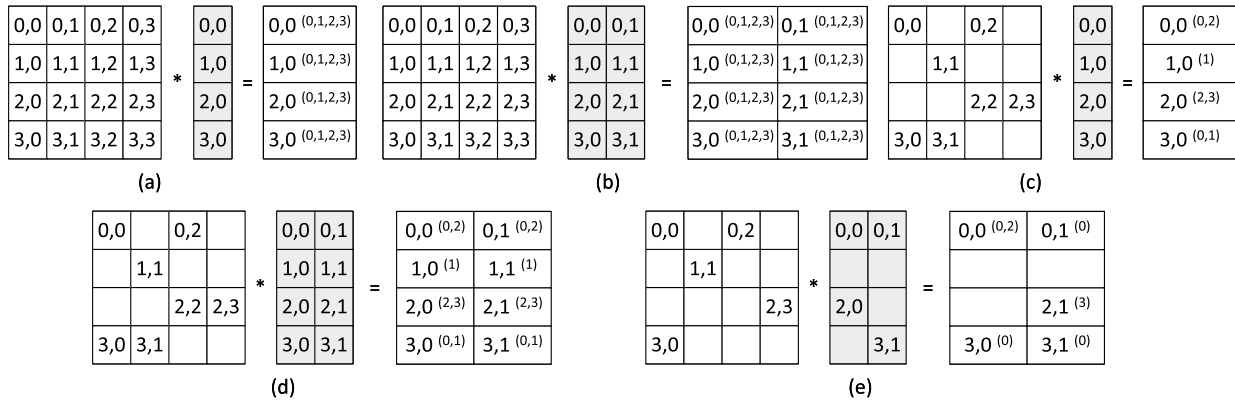


Fig. 3. Five operator examples. (a) Dense MV. (b) Multiplication of two dense matrices. (c) SpMV. (d) Sparse matrix-dense MM. (e) SpMspm.

sparse matrices—the former showing sparse-dense MM and the latter demonstrating sparse-SpMM—both cases displaying significantly reduced partial sum quantities, with Fig. 3(e)’s result matrix exhibiting the sparsest nonzero element distribution. Based on these representative computational patterns, subsequent analysis will thoroughly examine the execution mechanisms and performance variations of VersaAccel’s four configurations across these example computing tasks.

A. DMR Configuration

As shown in Fig. 4(a), this configuration retains only the distribution network functionality of the first column while disabling all other distribution networks and the reduction network modules. This design enables direct input/output data transmission through the PE array, forming a systolic array-like computing architecture. Notably, the distribution network and multiplication network employ implicit connections to facilitate horizontal data flow, while the multiplication network and reduction network similarly use implicit connections to support vertical partial-sum reduction operations. In this configuration, the system requires two data inputs per cycle to sustain computation.

For MV, as illustrated in Fig. 4(b), the original matrix A is partitioned into 2×2 subblocks for processing. During

computation, the first matrix subblock is initially preloaded into the PE array, followed by the elements of vector B flowing through the computing units in a systolic manner. After completing the first phase of computation, the system needs to replace the matrix subblock and continue processing the remaining elements of vector B. While this computational approach ensures functional correctness, it suffers from low array utilization and frequent data replacement issues, with the entire process consuming a total of 20 clock cycles.

For MM, the system employs a distinct blocking strategy. As shown in Fig. 4(c), the 4×4 matrix A is partitioned into two 4×2 subblocks, while the 4×2 matrix B is divided into two 2×2 subblocks. During computation, the subblocks of matrix B are first preloaded, followed by the corresponding subblocks of matrix A flowing in. After completing the initial computation, the system replaces the second subblock of matrix B to continue operations. Compared with MV, this computational pattern demonstrates higher efficiency, requiring only 16 clock cycles to complete the entire operation.

For sparse matrix operations, the DMR configuration employs a uniform dense computation approach. Both SpMV [see Fig. 4(d)] and sparse matrix-MM [see Fig. 4(e) and (f)] are executed following their corresponding dense matrix operation methods. While this design ensures functional compatibility, it inevitably leads to computational resource wastage when

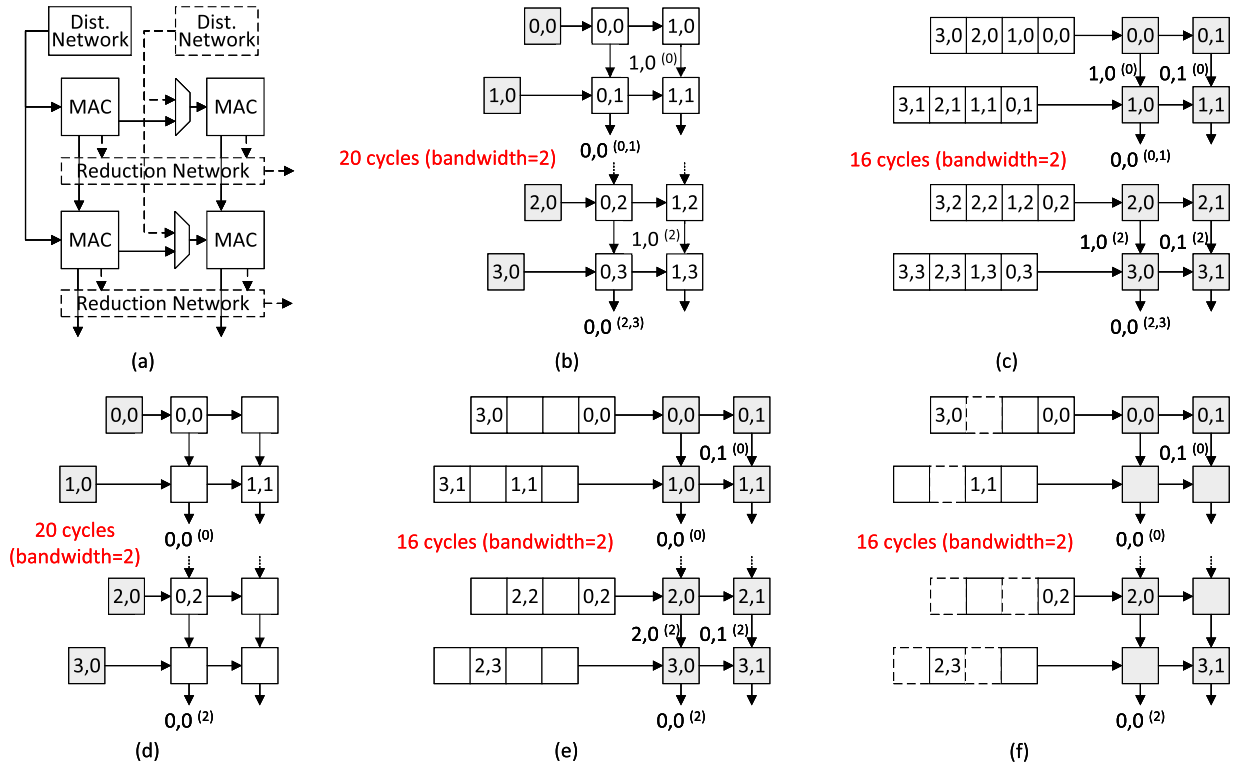


Fig. 4. Under the DMR configuration, the data flow mapping approach of VersaAccel for different operators. (a) DMR. (b) MV dataflow. (c) MM dataflow. (d) SpMV dataflow. (e) SpMM dataflow. (f) SpMSpM dataflow.

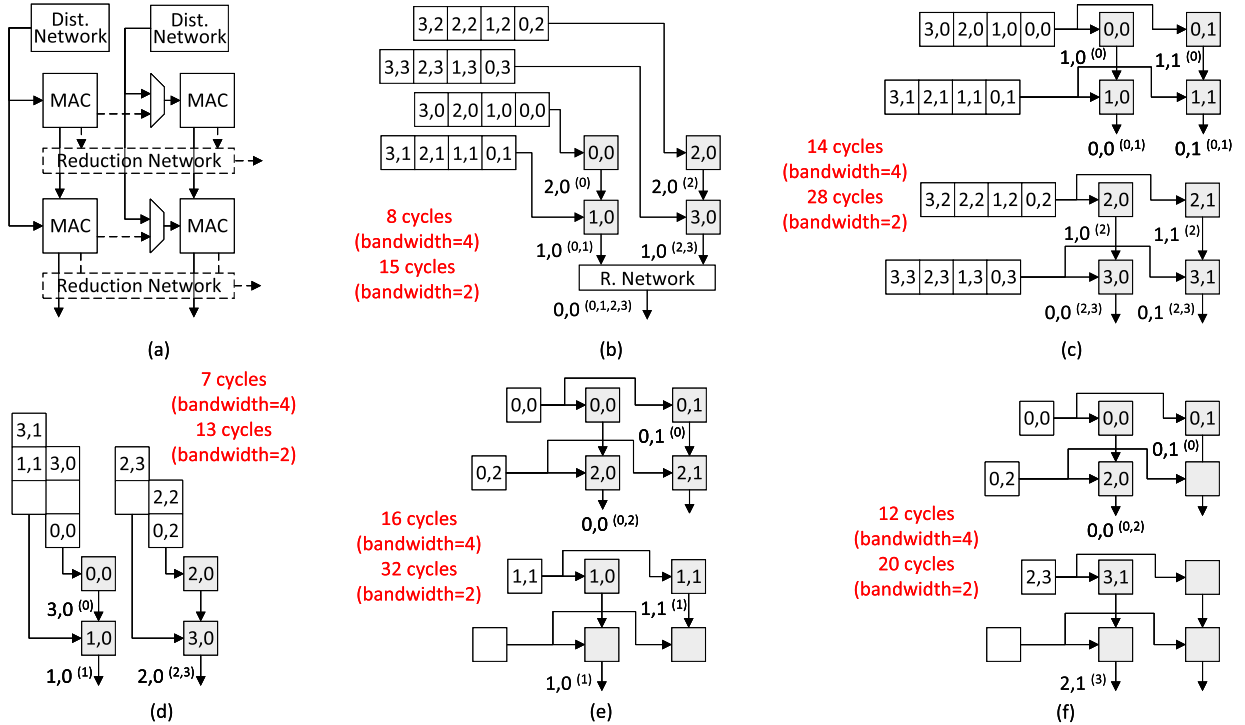


Fig. 5. Under the D-MR configuration, the data flow mapping approach of VersaAccel for different operators. (a) D-MR. (b) MV dataflow. (c) MM dataflow. (d) SpMV dataflow. (e) SpMM dataflow. (f) SpMSpM dataflow.

processing sparse data, with the two sparse operations requiring 20 and 16 clock cycles to complete, respectively.

B. D-MR Configuration

As illustrated in Fig. 5(a), the D-MR architecture reconfigures the connection scheme between the distribution network

and multiplication network, establishing VersaAccel in an explicit connection mode. In this configuration, input data is transmitted through the distribution network while partial sums undergo vertical reduction via PEs, forming two systolic array structures of 2×1 scale. The system's input bandwidth requirement is set at 4 data/cycle. When available bandwidth

decreases to 2 data/cycle, priority is given to maintaining the normal operation of a single column array.

In the MV scenario [see Fig. 5(b)], vector B is preloaded into the PE array, while column data from matrix A is transmitted through the distribution network in a systolic manner to ensure effective column-wise reduction. To enhance dataflow efficiency, the bottom-row reduction network is activated to enable direct merging of partial sums. When the system bandwidth reaches 4 data/cycle, the complete computation requires 8 clock cycles; when bandwidth is constrained to 2 data/cycle, the computation extends to 15 cycles.

For MM operations, a block processing strategy is adopted: matrix A is partitioned into two 4×2 subblocks, while matrix B is similarly divided into two 2×2 subblocks. As shown in Fig. 5(c), after preloading the first subblock of matrix B, the first column of matrix A simultaneously flows into both the first and second columns of the array, with the second column data following after a one-clock-cycle delay. This systolic transmission mechanism on a single column ensures effective implementation of column-wise reduction, requiring 14 and 28 computation cycles under sufficient bandwidth (4 data/cycle) and constrained conditions (2 data/cycle), respectively.

The SpMV [see Fig. 5(d)] employs a mapping method similar to dense MV but incorporates row compression optimization that eliminates all-zero rows in matrix A's subblocks, specifically compressing the submatrix flowing into the first array column to 3 rows and the second column's submatrix to 2 rows. By only removing entirely zero rows, the column-wise reduction process remains unaffected while computation cycles are reduced to 7 cycles at 4 data elements/cycle bandwidth and 13 cycles at 2 data elements/cycle bandwidth, achieving improved efficiency without compromising correctness through selective elimination of unnecessary zero-value computations.

For SpMM [see Fig. 5(e)], the Gustavson row-wise multiplication strategy is adopted: each nonzero row of matrix A is selected, and corresponding rows from matrix B are fetched for computation based on column indices. In the illustrated example, the first nonzero row of matrix A contains elements at positions (0, 0) and (0, 2), prompting the preloading of rows 1 and 3 from matrix B into the computing array. The nonzero values are then fed in systolic fashion to complete the computation. Although the second nonzero row of matrix A contains only one nonzero element (requiring loading of just one corresponding row from matrix B, resulting in suboptimal array utilization), the correctness of column-wise reduction is guaranteed, with the computation requiring 16 cycles (sufficient bandwidth) or 32 cycles (bandwidth-constrained).

Fig. 5(f) demonstrates SpMSpM, which extends the mapping method from Fig. 5(e) with additional optimizations based on the nonzero pattern of matrix B. When the row of matrix B corresponding to the nonzero elements in the second nonzero row of matrix A contains all zeros (causing invalid multiplications), that row is excluded from being loaded into the PE array. While this strategy further eliminates redundant computations, it still results in underutilization of computing resources to maintain reduction integrity. The final execution

cycles are 12 (with sufficient bandwidth) and 20 (under bandwidth-constrained conditions).

C. DM-R Configuration

The DM-R architecture employs a hybrid connection scheme, maintaining implicit systolic connections between the distribution network and multiplication network while adopting explicit connections between the multiplication network and reduction network. As shown in Fig. 6(a), this design enables PEs to focus solely on multiplication operations while the reduction network independently handles accumulation functions. In this configuration, each row of the array forms an independent computing unit without interrow data exchange, requiring an input bandwidth of 2 data/cycle.

The process of MV is illustrated in Fig. 6(b). Vector B is first preloaded into the PE array, followed by row-wise distribution of matrix A's row data through the distribution network. Since data transmission is accomplished via PEs, multiplication operations are only triggered when data reaches predetermined positions, ensuring computational correctness. This scheme requires 17 clock cycles to complete the full computation.

For MM, as shown in Fig. 6(c), the system decomposes the operation into a sequence of multiple MVs, following the same mapping approach as in Fig. 6(b). After completing the computation for each column of matrix B, the next column's data is loaded into the array. This iterative computation method ultimately requires 33 cycles to complete the entire operation.

The sparse operator computations uniformly adopt the Gustavson mapping strategy, processing nonzero rows of matrix A sequentially with row-level granularity. When encountering rows exceeding the array capacity, a batch loading mechanism ensures completion of current row processing before advancing. As demonstrated in Fig. 6(d) for SpMV, the system preloads matrix A's nonzero elements, multiplies them with corresponding vector B elements selected by column indices, and merges results through the reduction network, requiring exactly 9 cycles to complete the entire row-by-row computation process while maintaining immediate row transition after each completion.

Fig. 6(e) illustrates the computation flow for SpMM. After loading nonzero rows of matrix A, the system sequentially streams matching data from matrix B through the array for computation. This streaming processing approach achieves a total execution time of just 13 cycles.

For SpMSpM [see Fig. 6(f)], the system performs nonzero element intersection detection during the loading phase, computing only valid multiplications where both operands contain nonzero values. By skipping invalid computation rows—such as directly ignoring the second row of matrix A in the illustration—the total execution time is further optimized to just 7 cycles.

D. D-M-R Configuration

The D-M-R architecture enhances computational flexibility through its fully explicit connection design, as illustrated in Fig. 7(a). This configuration establishes direct connections

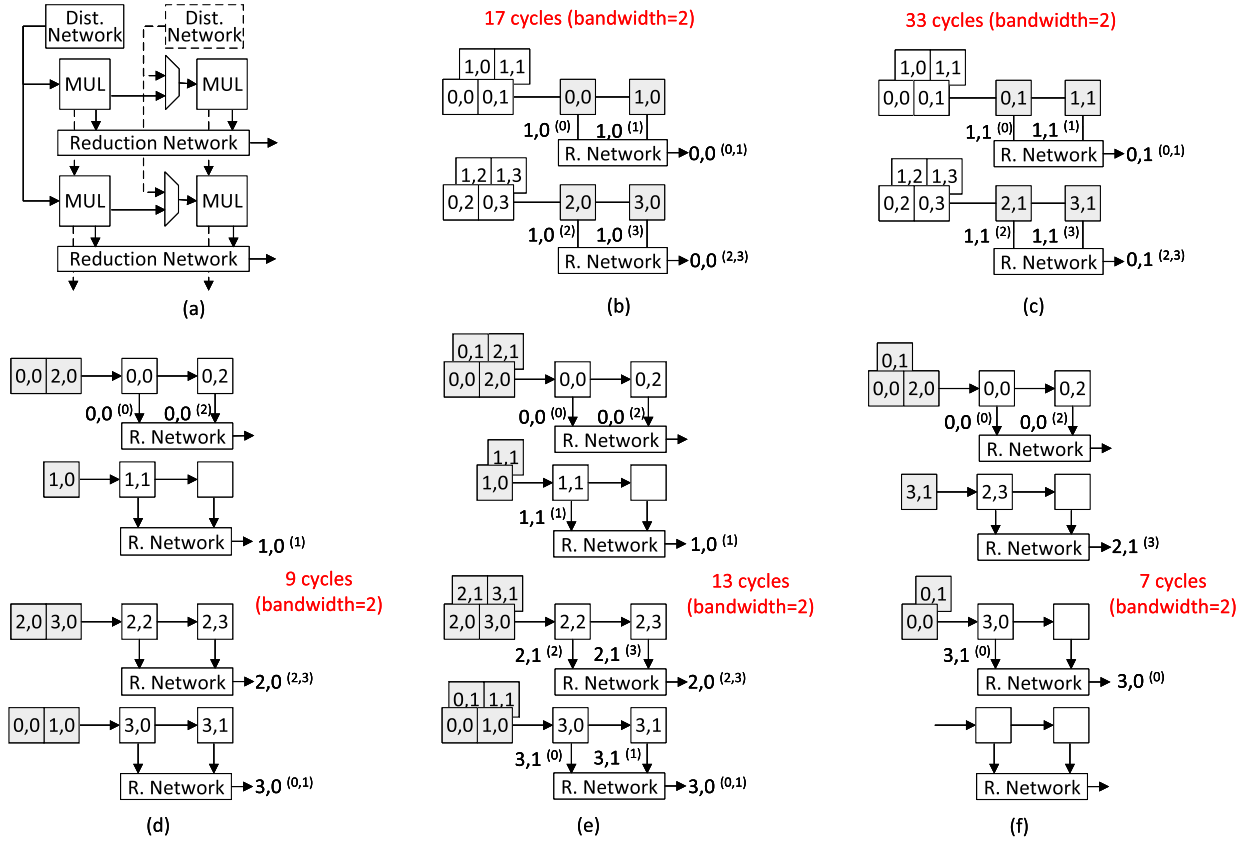


Fig. 6. Under the DM-R configuration, the data flow mapping approach of VersaAccel for different operators. (a) D-MR. (b) MV dataflow. (c) MM dataflow. (d) SpMV dataflow. (e) SpMM dataflow. (f) SpMSpM dataflow.

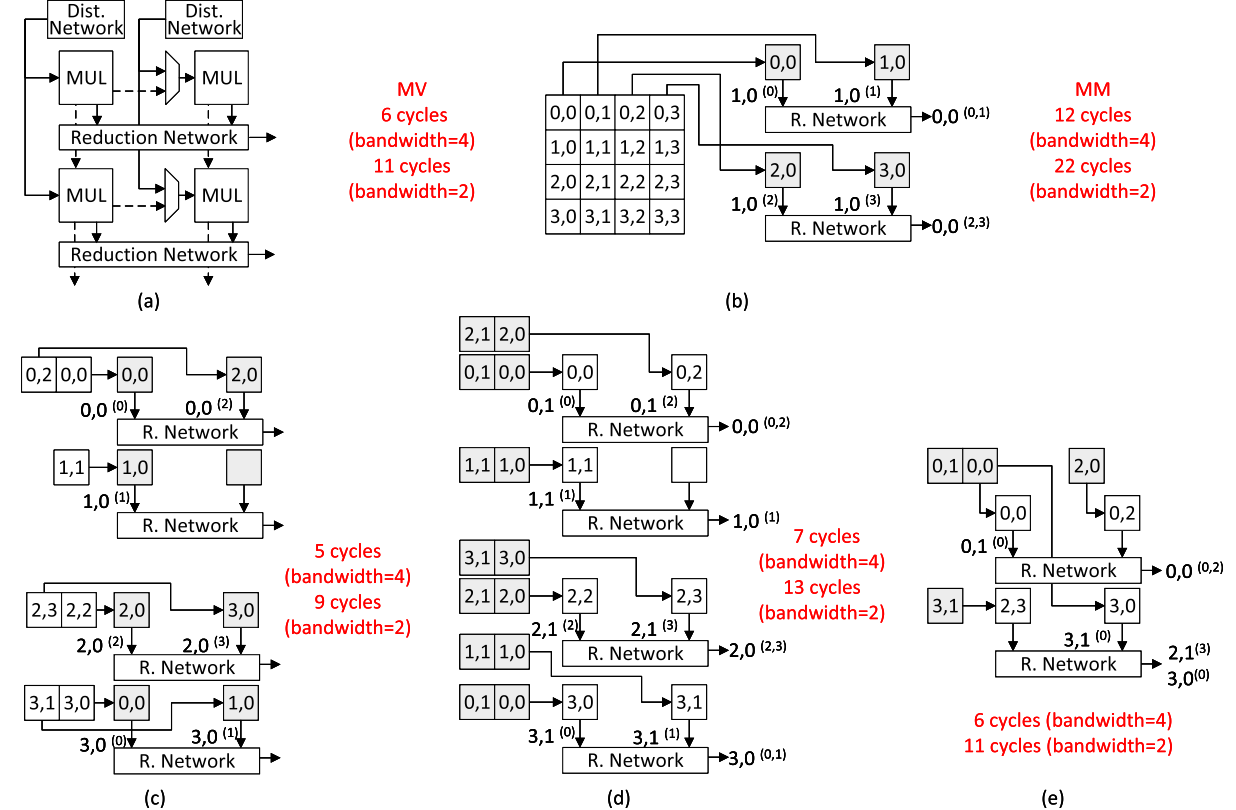


Fig. 7. Under the D-M-R configuration, the data flow mapping approach of VersaAccel for different operators. (a) D-M-R. (b) MV/MM dataflow. (c) SpMV dataflow. (d) SpMM dataflow. (e) SpMSpM dataflow.

between the distribution network, multiplication network, requirement of 4 data/cycle while also supporting a and reduction network, maintaining a standard bandwidth bandwidth-constrained mode of 2 data/cycle.

For dense matrix operations shown in Fig. 7(b), the system uses a unified mapping approach that loads columns of vector/matrix B first, followed by row-wise transmission of matrix A, achieving full data reuse and completing MV in 6 cycles (12 cycles when bandwidth-constrained) and matrix-MM in 12 cycles (22 cycles when bandwidth-constrained) through its optimized reduction network architecture.

The SpMV in Fig. 7(c) implements a row-level parallel strategy that first identifies valid data through intersection operations, preloads vector B, then distributes matrix A's nonzero elements to corresponding processing units, completing in just 5 cycles (4 data/cycle) or 9 cycles (2 data/cycle) by maximizing data reuse during the staged computation process.

The SpMM implementation in Fig. 7(d) employs an enhanced Gustavson mapping method where each array row independently processes one nonzero row computation—first loading nonzero elements from matrix A followed by streaming corresponding row data from matrix B, achieving execution times of 7 cycles (at 2 data/cycle bandwidth) and 13 cycles (at 4 data/cycle bandwidth) through this pipelined row-parallel approach.

Fig. 7(e) demonstrates the optimized SpMSpM implementation that enhances the Gustavson method by consolidating multiple nonzero rows of matrix A into single-row processing units while utilizing the reduction network to maintain computational independence, effectively eliminating resource idle time. This improved design reduces execution time to 6 cycles (at 4 data/cycle bandwidth) and 11 cycles (at 2 data/cycle bandwidth), achieving significantly higher hardware resource utilization through its innovative row-merging computation paradigm.

E. Execution Analysis

Table II provides a detailed comparison of the execution time performance for the matrix operators shown in Fig. 3 across four different architectural configurations. By analyzing computational efficiency under both sufficient and constrained bandwidth conditions, we observe distinct performance characteristics exhibited by each configuration scheme.

The DMR configuration achieves data movement through a systolic array implementation by disabling partial distribution and reduction networks. This configuration demonstrates excellent performance under bandwidth-constrained conditions, where its lower input bandwidth requirements and efficient data flow mechanisms ensure stable computational performance. However, when additional bandwidth becomes available, its performance improvement is limited due to the inability to effectively utilize extra bandwidth resources. Moreover, it shows poor performance when handling sparse matrices due to a lack of flexibility.

The D-MR configuration significantly improves performance under sufficient bandwidth conditions by enhancing distribution network flexibility. For instance, execution times for MV and SpMV are reduced to 8 and 7 cycles, respectively. However, when available bandwidth is halved, approximately half of the PEs in the computing array become idle, directly resulting in reduced computational throughput. This unbalanced resource utilization leads to doubled operator execution

TABLE II
EXECUTION TIMES FOR REPRESENTATIVE MATRIX OPERATORS UNDER DIFFERENT CONFIGURATIONS

Cycles (4/2 data/cycle)	DMR	D-MR	DM-R	D-M-R
MV	20 / 20	8 / 15	17 / 17	6 / 11
MM	16 / 16	14 / 28	33 / 33	12 / 22
SpMV	20 / 20	7 / 13	9 / 9	5 / 9
SpMM	16 / 16	16 / 32	13 / 13	7 / 13
SpMSpM	16 / 16	12 / 20	7 / 7	6 / 11

times, reflecting the configuration's sensitivity to bandwidth variations.

The DM-R configuration employs explicit connections between multiplication and reduction networks to achieve row-wise computational independence, giving it advantages in sparse matrix operations. Since this configuration uses a fixed low-bandwidth design, it maintains stable computational performance even in bandwidth-constrained environments. Taking SpMSpM as an example, execution time can be optimized to 7 cycles, demonstrating the architecture's efficiency in processing sparse data. However, this row-independent computation mode also introduces limitations in data reuse, resulting in relatively longer execution times for matrix-MM which becomes the primary performance bottleneck.

The D-M-R configuration adopts a full connectivity design through explicit connections between distribution, multiplication, and reduction networks, achieving maximum computational flexibility. This configuration demonstrates excellent computational performance in high-bandwidth environments, where MV completes in just 6 cycles, showing significant advantages over other configurations. Even under bandwidth-constrained conditions, this architecture maintains relatively good computational efficiency, controlling matrix-MM execution time within 22 cycles, outperforming most comparable designs.

The performance differences between configurations essentially stem from their distinct design philosophies: simplified network connections are more suitable for resource-constrained environments, while enhanced flexibility solutions perform better when bandwidth is sufficient. Selecting the optimal configuration requires comprehensive consideration of target workload characteristics and system resource conditions to find the best balance between computational efficiency and power consumption. Although the D-M-R configuration leads in computing speed, its higher hardware complexity may result in suboptimal energy efficiency, a characteristic that requires special attention in power-sensitive application scenarios.

IV. COST MODEL FOR VERSAACCEL

We propose a cost model designed to ascertain the optimal configuration of VersaAccel under varying operators and system bandwidth conditions. The model aims to accelerate five algebraic operators: MM, MV, SpMV, SpMM, and SpMSpM. VersaAccel supports four distinct configurations (C): DMR, D-MR, DM-R, and D-M-R. Each configuration is capable of executing the aforementioned operators, albeit with differing performance characteristics. Given a specific operator (S) and

system memory bandwidth (BW), the objective of the cost model is to determine the configuration (C^*) that minimizes the execution time (T).

A. Problem Definition

The optimization problem for the cost model is formally defined as follows. Specifically, the input consists of two key parameters: the operator type S , where $S \in \{MM, MV, SpMV, SpMM, SpMSPM\}$, and the system memory bandwidth BW, measured in “elements per cycle.” On the other hand, the output is the optimal configuration C^* , which belongs to the set of all possible configurations $C_{all} = \{DMR, D-MR, DM-R, D-M-R\}$. The primary objective of the model is to minimize the total cost $Cost(S, C, BW)$ by selecting the configuration C^* that achieves the lowest execution time for the given operator and system bandwidth. This can be mathematically expressed as

$$C^* = \arg \min_{C \in C_{all}} Cost(S, C, BW).$$

By doing so, the formulation ensures an efficient mapping of computational tasks to hardware resources, thereby optimizing performance across diverse algebraic operators and system conditions. Overall, this approach provides a systematic framework for achieving optimal computational efficiency in varying scenarios.

B. Cost Model

The cost model divides the total cost $Cost(S, C, BW)$ into two components: the computation cost ($Cost_{comp}$) and the memory access cost ($Cost_{mem}$), which represent the time costs associated with computation and data transfer, respectively. The total cost is expressed as

$$\begin{aligned} Cost(S, C, BW) \\ = Cost_{comp}(S, C, BW) + Cost_{mem}(S, C, BW). \end{aligned}$$

1) $Cost_{Comp}$: The computation cost $Cost_{comp}(S, C, BW)$ represents the number of computation cycles required to execute operator S under configuration C . This cost depends on the accelerator’s hardware design, including pipeline depth, parallelism, and hardware resource allocation. The computation cost is also influenced by the system memory bandwidth. When the bandwidth cannot meet the data consumption rate, some computation units may stall, leading to a phenomenon known as “bandwidth starvation.”

2) $Cost_{Mem}$: The memory access cost $Cost_{mem}(S, C, BW)$ represents the time required to transfer data from memory to the accelerator under configuration C . This cost depends on the system memory bandwidth (BW) and the amount of data accessed by operator S under configuration C . The memory access cost is expressed as

$$Cost_{mem}(S, C, BW) = \frac{Data(S, C)}{BW}$$

where $Data(S, C)$ denotes the total amount of data accessed by operator S under configuration C (measured in elements).

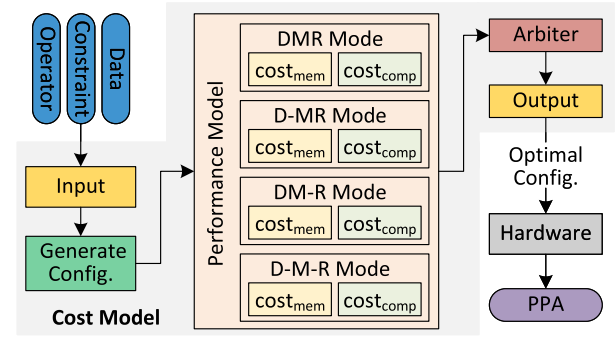


Fig. 8. Workflow of cost model. In response to different input scenarios, the model precomputes the costs for various modes, selects the configuration most suitable for the current input, and outputs it to the hardware design.

C. Optimal Configuration Selection

The cost model combines the computation cost and memory access cost to determine the optimal configuration C^* . Specifically, the model enumerates all configurations, calculates the total cost for each configuration, and selects the one with the minimum cost

$$\begin{aligned} C^*(S, BW) = \arg \min_{C \in C_{all}} [& Cost_{comp}(S, C, BW) \\ & + Cost_{mem}(S, C, BW)]. \end{aligned} \quad (1)$$

D. Cost Model Workflow

The workflow of the proposed cost model is illustrated in Fig. 8, which begins by taking the target operator S , system constraints, and data characteristics as inputs to generate candidate configurations C . The performance model then evaluates each configuration across different processing modes to precompute the computational cost $Cost_{comp}(S, C, BW)$ and memory access volume $Data(S, C)$ based on dataflow mapping principles where bandwidth BW directly determines execution time, instead of adjusting computation cycles based on the ratio to ideal bandwidth. These costs are aggregated in the total cost calculation $Cost(S, C, BW)$, followed by the arbitration module selecting the optimal configuration $C^* = \arg \min Cost(S, C, BW)$ that is deployed to hardware to realize the final power, performance, and area outcomes.

The cost model provides a systematic methodology for determining the optimal configuration of a custom accelerator. It explicitly accounts for both computation cost and memory access cost, revealing the tradeoff relationship between the two. Notably, the dual impact of memory bandwidth on execution time is explicitly modeled as follows.

- 1) *Impact on Computation Cost*: When bandwidth is insufficient, computation units may not be fully utilized due to inadequate data supply, leading to an increase in computation cost.
- 2) *Impact on Memory Access Cost*: Bandwidth directly determines the speed of data transfer between memory and the accelerator, thereby influencing the memory access cost. By leveraging this model, the optimal configuration can be rapidly selected under varying operator and bandwidth conditions, enabling efficient utilization of accelerator resources. This approach ensures a

balanced optimization of computational and memory-related performance, enhancing the overall efficiency of the accelerator in diverse scenarios.

E. Discussion on Configuration Switching

A valid concern for our configurable architecture is the latency overhead when switching configurations. We emphasize that VersaAccel's reconfiguration—achieved by loading control words to enable prewired datapaths—incurs negligible one-time overhead. This cost is easily hidden by overlapping it with necessary software operations (e.g., driver calls or DMA setup) and is fully amortized by the significant performance gains achieved during the subsequent execution of the operator, which typically spans millions of cycles. Thus, the reconfiguration process does not impact the critical path and poses no practical performance penalty.

V. METHODOLOGY

A. System

We configured VersaAccel with an 8×8 array structure, where the distribution network is organized into eight groups of eight-input subdistribution networks, with each group responsible for data input to a single column of the array. Similarly, eight reduction networks, each with eight inputs, are deployed to handle data output from each row of the array. The input buffer and output buffer are sized at 256 and 64 KB, respectively, with eight banks for parallel access. The off-chip memory is configured with 8 HBM channels, each 64 bits wide, providing a total bandwidth of 128 GB/s. Using Design Compiler with 7-nm technology [20], we synthesized the RTL design, achieving an operating frequency of 1 GHz.

B. Baseline

To evaluate the performance of VersaAccel, we selected several state-of-the-art array accelerators designed for different operators as baselines for comparison. In addition to their specifically optimized operators, we also mapped other operators to these accelerators for experimentation, aiming to analyze and compare the design challenges posed by multiple operators. The accelerators selected for comparison include TPU [7], STPU [9], SIGMA [8], Tensaurus [12], and SPSA [10]. We simulated the data flow of these designs to establish a simulator for evaluating the number of execution cycles.

To ensure fairness in the comparison, we standardized the number of compute units across all accelerators to 64. Specifically, TPU, STPU, and SIGMA, which originally featured a 128×128 array, were scaled down to an 8×8 array for our experiments. Similarly, SPSA was configured to the same scale, aligning it with Tensaurus and VersaAccel. In addition, the on-chip buffer size for all designs was constrained to 512 KB, with the buffer allocations proportionally divided according to each design's requirements. All designs were set to operate at 1 GHz, with an off-chip HBM bandwidth of 128 GB/s. All computations were performed using 32-bit precision.

The execution cycle counts primarily focus on on-chip processing, including input reading, computation, and output

TABLE III
AREA AND POWER CONSUMPTION OF VERSAACCEL AND OTHER BASELINE ACCELERATORS

Component	Area / mm ²	Power / W
TPU	0.146	0.437
SIGMA	0.183	0.529
STPU	0.152	0.543
SPSA	0.158	0.552
Tensaurus	0.275	0.672
VersaAccel	0.190	0.584

storage. Other overheads, such as data preprocessing and format encoding/decoding, were temporarily excluded from the analysis.

C. Datasets

The experiments utilized datasets from the following collections.

1) *CNN Collection*: Most operations in CNN models are convolutions, which we transformed into MM operations using im2col. These serve as the test set for MM. Through pruning, we obtained weight matrices with sparsity levels ranging from 0.5 to 0.9, which are used as the test set for SpMM. By applying the ReLU operation, we generated feature matrices with sparsity levels between 0.5 and 0.7, which are used as the test set for SpMSPM. We processed models such as VGG16 [21] and ResNet18 [22] layer by layer to generate these datasets.

2) *Transformer Collection*: Feedforward neural networks and self-attention mechanisms involve a large number of MM and MV operations. We applied magnitude-based pruning to adjust the sparsity of weight matrices to 0.5–0.8, creating test sets for SpMM and SpMV. Models such as LLaMA2-7B [23] and BERT-base [5] were selected for analysis and processing.

3) *SuiteSparse Collection*: SuiteSparse [24] is a widely used collection of sparse matrices from real-world applications, including scientific computing, engineering, economics, and social networks. We randomly selected 12 sparse matrices from this collection. For SpMV, the matrices were multiplied by randomly generated dense vectors. For SpMM, the matrices were multiplied by randomly generated matrices with 128, 256, and 512 columns. For SpMSPM, the matrices were multiplied by their transposed counterparts.

VI. EVALUATION

A. Area and Power

We evaluated the area and power consumption of VersaAccel and all baseline accelerators, with the results summarized in Table III. Fabricated using a 7-nm process, VersaAccel occupies an area of 0.19 mm² (with distribution network: 0.006 mm², multiplication network: 0.043 mm², reduction network: 0.012 mm², and buffer: 0.129 mm²), which is 1.30×, 1.04×, 1.25×, 1.2×, and 0.69× the area of TPU, SIGMA, STPU, SPSA, and Tensaurus, respectively. While VersaAccel and Tensaurus, which support multiple operators, do not exhibit an overall area advantage compared to accelerators optimized for specific operators, VersaAccel achieves a smaller area than Tensaurus. This is primarily due to Tensaurus's

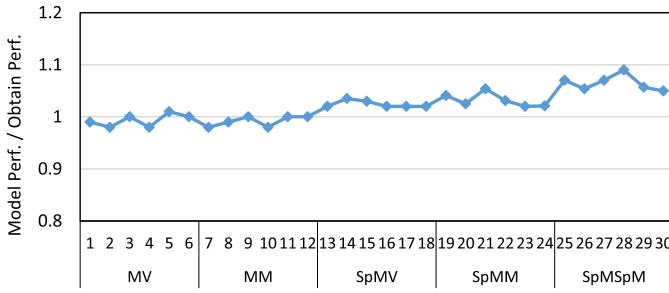


Fig. 9. Comparison between the cost model and achieved performance shows a discrepancy due to potential computation and memory access congestion during actual execution.

use of a larger number of scratchpad memories. In terms of total power consumption, VersaAccel consumes 0.584 W, which is $1.34\times$, $1.10\times$, $1.08\times$, $1.06\times$, and $0.87\times$ the power consumption of TPU, SIGMA, STPU, SPSA, and Tensaurus, respectively.

B. Cost Model Performance

We compared the performance predicted by the cost model with the actual performance achieved by VersaAccel to evaluate the model's accuracy. As shown in Fig. 9, we selected 30 test cases, with six cases for each operator type.

For MV and MM operators, the error remains within 3%, primarily because the regular patterns in dense data movement and computation allow the cost model to achieve high evaluation accuracy.

When matrices become sparse, data reuse decreases while memory access demands increase, making computation and memory access more prone to congestion and stall conditions. In SpMV and SpMM operators, when only one matrix is sparse, the cost model tends to overestimate the achieved performance, with errors increasing to 5%.

This effect becomes more pronounced in SpMSpM operators where both matrices are sparse, with errors further increasing to nearly 10%. Overall, the cost model's performance predictions reliably reflect actual performance trends.

C. VersaAccel Performance Under Four Configurations

We first evaluated the performance of VersaAccel under its four configurations. For each test, the performance of all four configurations was normalized, and the geometric mean of the normalized performance for each configuration was calculated as the final result. Since different configurations activate different modules, resulting in varying power consumption, we further adjusted the evaluation metric to power efficiency. To assess the impact of bandwidth, we conducted tests under two bandwidth conditions: 64 elements/cycle and 8 elements/cycle.

Under high bandwidth conditions [see Fig. 10(a)], the D-M-R configuration achieved the best performance for all operators except MM, where DMR performed the best. Although D-M-R consumes the most power, its flexible processing capabilities result in higher power efficiency. However, as shown in Fig. 10(b), under low bandwidth conditions, the performance of D-MR and D-M-R configurations degrades significantly.

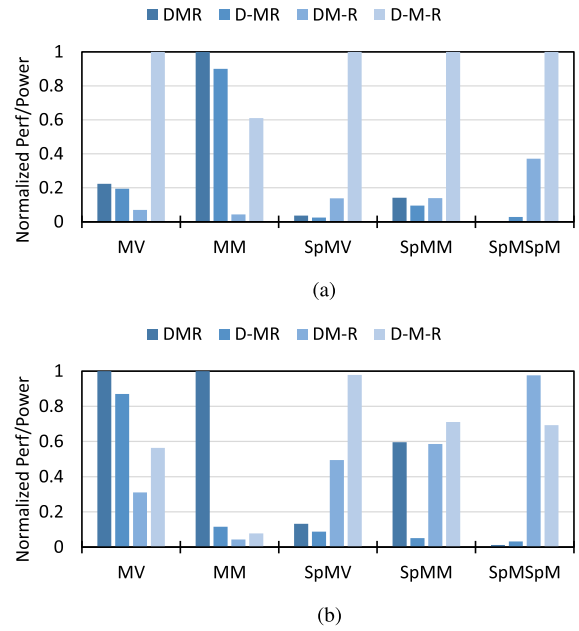


Fig. 10. Normalized performance/power comparison of VersaAccel executing five operators under its four configurations. Under different bandwidth conditions, the performance of each configuration varies significantly. (a) Bandwidth is set to 64 elements/cycle. (b) Bandwidth is set to 8 elements/cycle.

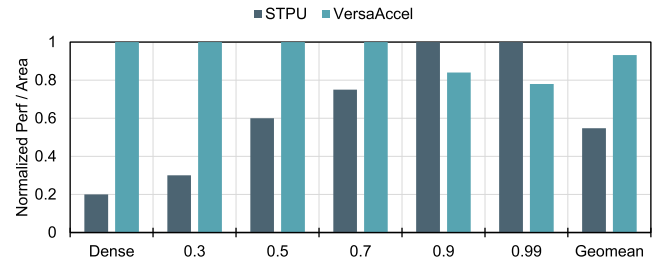


Fig. 11. Performance comparison on MV/SpMV.

The distribution network cannot meet the computational demands, leading to an overall performance drop. In contrast, the DMR and DM-R configurations, which have lower bandwidth requirements, maintain consistent performance even under constrained bandwidth conditions, as their implicit connections and reduced dependency on explicit data transfers make them less susceptible to bandwidth limitations.

These results highlight the tradeoffs between flexibility and bandwidth sensitivity in VersaAccel's configurations. While D-M-R excels in high-bandwidth scenarios, DMR and DM-R demonstrate robustness in low-bandwidth environments, making them suitable for resource-constrained applications.

D. Specific Operator Performance

We evaluated individual operators by comparing VersaAccel with baseline accelerators specifically optimized for those operators, using area efficiency (performance/area) as the metric.

1) *MV and SpMV*: We compared VersaAccel with STPU, testing matrices with sparsity levels of 0 (dense), 0.3, 0.5, 0.7, 0.9, and 0.99, multiplied by dense vectors. As shown in Fig. 11, STPU performs better at very high sparsity levels

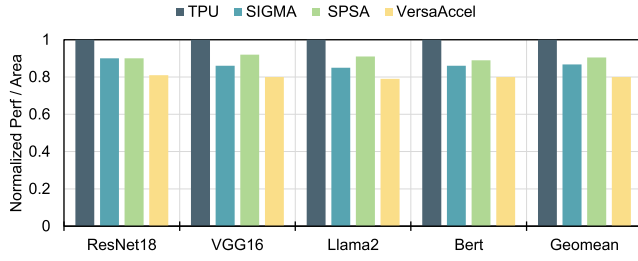


Fig. 12. Performance comparison on MM.

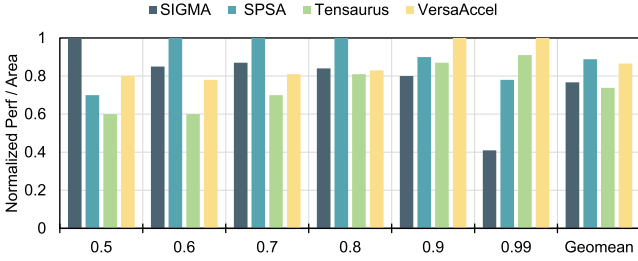


Fig. 13. Performance comparison on SpMM.

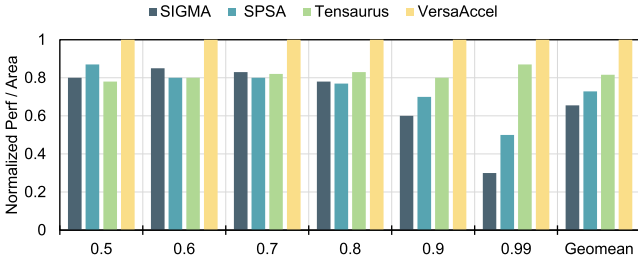


Fig. 14. Performance comparison on SpMSpM.

(>0.9), while VersaAccel outperforms in other sparsity levels and dense scenarios. Overall, VersaAccel achieves a $1.71\times$ improvement over STPU across the entire test range.

2) *MM*: We compared VersaAccel with TPU, SIGMA, and SPSA, using MMs extracted from ResNet18, VGG16, LLaMA2-7B, and BERT-base. As shown in Fig. 12, the performance gap among the four accelerators is small, with differences primarily due to area overhead. TPU, with its streamlined architecture, achieves the highest area efficiency, while VersaAccel, designed to support multiple operators, has the largest area overhead among the four.

3) *SpMM*: We compared VersaAccel with SIGMA, SPSA, and Tensaurus, testing matrices with sparsity levels of 0.5, 0.6, 0.7, 0.8, 0.9, and 0.99. As shown in Fig. 13, the best-performing accelerator varies with sparsity levels: SIGMA excels at low sparsity (0.5), SPSA performs best at medium sparsity (0.6–0.8), and VersaAccel dominates at high sparsity (≥ 0.9). Overall, VersaAccel achieves $1.13\times$, $0.97\times$, and $1.17\times$ improvements over SIGMA, SPSA, and Tensaurus, respectively.

4) *SpMSpM*: As shown in Fig. 14, VersaAccel outperforms SIGMA, SPSA, and Tensaurus across all tested sparsity levels, achieving $1.53\times$, $1.37\times$, and $1.23\times$ improvements, respectively.

These results highlight VersaAccel's versatility and efficiency across a wide range of operators and sparsity levels,

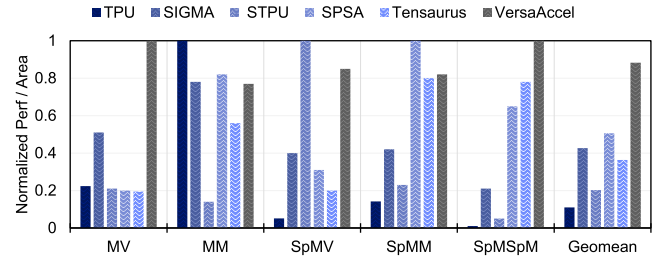


Fig. 15. Comparison of the performance of all accelerators when calculating different operators.

demonstrating its ability to adapt to diverse computational demands while maintaining competitive performance.

E. Multiple Operators Performance

We conducted tests on VersaAccel and five other baseline designs across different operators, evaluating their performance on MV, MM, SpMV, SpMM, and SpMSpM operations under various datasets. For a fair comparison, we used performance/area as the metric to further assess their area efficiency. In addition, we normalized the performance relative to the best-performing accelerator to facilitate analysis. The experimental results are shown in Fig. 15. Overall, VersaAccel's area efficiency is $8.02\times$, $2.07\times$, $4.37\times$, $1.75\times$, and $2.43\times$ that of TPU, SIGMA, STPU, SPSA, and Tensaurus, respectively.

Specifically, in MV, VersaAccel benefits from its flexible distribution and reduction networks, enabling efficient mapping of vector data. In contrast, other designs suffer from computational resource underutilization due to dataflow limitations, resulting in poorer performance. In MM, TPU achieves the best performance due to its simpler design and lower area overhead, while STPU and Tensaurus lag behind due to excessive resource underutilization (STPU) and larger area overhead (Tensaurus). For SpMV, STPU, which is specifically optimized for this operator, performs exceptionally well, whereas SPSA and Tensaurus are limited by their architectural constraints, using only a single column of PEs for computation. In SpMM, SPSA takes the lead, while Tensaurus and VersaAccel trail due to their larger area overhead. In SpMSpM, VersaAccel performs the best, while SPSA suffers from performance degradation due to its focus on compressing only one side of the matrix.

On further analysis, Tensaurus employs substantial buffer resources that significantly impact area efficiency, coupled with a fixed hardware architecture that limits resource utilization despite supporting multiple dataflows—particularly evident in MV, where only single-column processing occurs while other arrays remain idle. In contrast, VersaAccel achieves superior performance/area efficiency through tunable hardware configurations, operator-specific dataflows, and a cost model that dynamically selects optimal execution schemes, enabling full resource utilization and peak performance maximization.

In summary, VersaAccel demonstrates strong performance across all five operators. SIGMA shows moderate performance, while SPSA and Tensaurus exhibit weaknesses in vector processing. TPU and STPU excel in MM and MV,

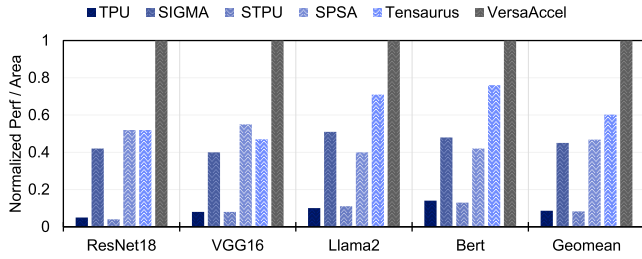


Fig. 16. Comparison of the performance of all accelerators when calculating different models.

respectively, but perform poorly on other operators. These results highlight VersaAccel’s versatility and efficiency in handling a wide range of computational tasks.

F. Model Performance

When testing the models, we extracted all matrix-matrix and matrix-vector operations from each model and executed them individually on the six accelerators. The results were then aggregated to obtain the final performance metrics. It is important to note that the initial models were dense, with little to no sparsity. To introduce sparsity, we applied pruning and ReLU operations to the weights and activations in CNN models, as well as to the weights, activations, and attention matrices in transformer-based models. We conducted unified tests using both the original dense models and sparse models with varying levels of sparsity.

The final experimental results, as shown in Fig. 16, are evaluated using the normalized performance/area metric. VersaAccel demonstrates superior performance across all four tested models, achieving $11.56\times$, $2.22\times$, $12.16\times$, $2.14\times$, and $1.66\times$ improvements over TPU, SIGMA, STPU, SPSA, and Tensaurus, respectively. These results underscore VersaAccel’s ability to efficiently handle both dense and sparse operations, making it a highly versatile and area-efficient accelerator for a wide range of machine learning models.

VII. RELATED WORK

The current landscape of spatial accelerators [2], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38] for sparse and dense matrix operators reveals several distinct design approaches. The distribution network, multiplication network, and reduction network are the three core components of these accelerators. Their interconnections define the application scenarios for different accelerators. For example, TPU [7] and MEISSA [39] are accelerators specifically designed for MM, where the distribution network is implicitly connected to the multiplication network, and the multiplication network is implicitly connected to the reduction network. This configuration is well-suited for regular MM operators. Building on this implicit connection, accelerators like STPU [9], Mentha [16], and SPSA [10] preprocess and compress sparse matrices to support SpMV and SpMM/SpMSPM. These accelerators retain the advantages of implicit connections while modifying the PE to handle sparse data. However, this approach is not ideal for sparse matrix operators that require flexible data fetching and matching.

Other works, such as SIGMA [8], Flexagon [17], Spada [19], and MatRaptor [3], have adopted explicit connections between the distribution network and multiplication network, as well as between the multiplication network and reduction network. This design significantly enhances the flexibility of data transfer, allowing for efficient handling of nonzero data matching and distribution. VersaAccel integrates the strengths of these designs by offering configurable connection schemes. It can adaptively select different configurations for different operators, thereby ensuring optimal acceleration for multiple operators. This flexibility in adjusting the interconnections of the core networks enables VersaAccel to achieve high performance across a wide range of matrix operations.

VIII. CONCLUSION

This article presents VersaAccel, a configurable accelerator design targeting various sparse and dense matrix operators. We adjusted the relationships between the distribution network, multiplication network, and reduction network to derive four different configuration schemes. We elaborated on the dataflow mapping schemes for different operators in each configuration and validated their rationality. In addition, we proposed a cost model that helps select the appropriate configuration for handling different operator types under various scenarios. Finally, through comprehensive experimental evaluations, compared with baseline accelerators, we achieved performance/area improvements ranging from $1.75\times$ to $8.02\times$ in multioperator tests and from $1.66\times$ to $11.56\times$ in model tests.

REFERENCES

- [1] S. E. Kurt, A. Sukumaran-Rajam, F. Rastello, and P. Sadayappan, “Efficient tiled sparse matrix multiplication through matrix signatures,” in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2020, pp. 1–14.
- [2] S. Pal et al., “OuterSPACE: An outer product based sparse matrix multiplication accelerator,” in *Proc. IEEE HPCA*, Feb. 2018, pp. 724–736.
- [3] N. Srivastava, H. Jin, J. Liu, D. H. Albonese, and Z. Zhang, “MatRaptor: A sparse-sparse matrix multiplication accelerator based on row-wise product,” in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2020, pp. 766–780.
- [4] J. Albericio, P. Judd, T. Hetherington, T. M. Aamodt, N. E. Jerger, and A. Moshovos, “Cnvlutin: Ineffectual-neuron-free deep neural network computing,” in *Proc. ACM/IEEE ISCA*, Jun. 2016, pp. 1–13.
- [5] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [6] G. Zhao, J. Lin, Z. Zhang, X. Ren, Q. Su, and X. Sun, “Explicit sparse transformer: Concentrated attention through explicit selection,” 2019, *arXiv:1912.11637*.
- [7] N. P. Jouppi et al., “In-datacenter performance analysis of a tensor processing unit,” in *Proc. ACM/IEEE 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, Toronto, ON, Canada, Jun. 2017, pp. 1–12.
- [8] E. Qin et al., “SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training,” in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 58–70.
- [9] X. He et al., “Sparse-TPU: Adapting systolic arrays for sparse matrices,” in *Proc. ICS*, 2020, pp. 19:1–19:12.
- [10] M. Tang, M. Wen, J. Yang, Z. Xue, and J. Shen, “SPSA: Exploring sparse-packing computation on systolic arrays from scratch,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 2, pp. 497–511, Feb. 2025.
- [11] Y. Yang, J. Emer, and D. Sánchez, “Trapezoid: A versatile accelerator for dense and sparse matrix multiplications,” in *Proc. ISCA*, Jul. 2024, pp. 931–945.

- [12] N. Srivastava, H. Jin, S. Smith, H. Rong, D. Albonesi, and Z. Zhang, "Tensaurus: A versatile accelerator for mixed sparse-dense tensor computations," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, San Diego, CA, USA, Feb. 2020, pp. 689–702.
- [13] A. Parashar et al., "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proc. ISCA*, Jun. 2017, pp. 27–40.
- [14] K. Hegde et al., "Extensor: An accelerator for sparse tensor algebra," in *Proc. MICRO*, 2019, pp. 319–333.
- [15] H. T. Kung, B. McDanel, and S. Q. Zhang, "Packing sparse convolutional neural networks for efficient systolic array implementations: Column combining under joint optimization," in *Proc. Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2019, pp. 821–834.
- [16] M. Tang et al., "Mentha: Enabling sparse-packing computation on systolic arrays," in *Proc. 51st Int. Conf. Parallel Process.*, Aug. 2022, pp. 1–11.
- [17] F. Muñoz-Martínez, R. Garg, M. Pellauer, J. L. Abellán, M. E. Acacio, and T. Krishna, "Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient DNN processing," in *Proc. ASPLOS*, 2023, pp. 252–265.
- [18] K. Zhong et al., "FEASTA: A flexible and efficient accelerator for sparse tensor algebra in machine learning," *ASPLOS*, pp. 349–366, 2024.
- [19] Z. Li et al., "Spada: Accelerating sparse matrix multiplication with adaptive dataflow," in *Proc. ASPLOS*, 2023, pp. 747–761.
- [20] V. Vashishtha, M. Vangala, and L. T. Clark, "ASAP7 predictive design kit development and cell design technology co-optimization: Invited paper," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2017, pp. 992–998.
- [21] Simonyan, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, 2015, pp. 1–14.
- [22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, Jun. 2016, pp. 770–778.
- [23] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [24] T. A. Davis and Y. Hu, "The university of Florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1–25, Nov. 2011.
- [25] Z. Zhang, H. Wang, S. Han, and W. J. Dally, "SpArch: Efficient architecture for sparse matrix multiplication," in *Proc. HPCA*, 2020, pp. 261–274.
- [26] X. Zhou et al., "Cambricon-S: Addressing irregularity in sparse neural networks through a cooperative software/hardware approach," in *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2018, pp. 15–28.
- [27] S. Han et al., "ESE: Efficient speech recognition engine with sparse LSTM on FPGA," in *Proc. FPGA*, 2016, pp. 75–84.
- [28] Y. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 367–379.
- [29] S. Han et al., "EIE: Efficient inference engine on compressed deep neural network," in *Proc. ISCA*, Jun. 2016, pp. 243–254.
- [30] G. Gerogiannis, S. Aananthakrishnan, J. Torrellas, and I. Hur, "HotTiles: Accelerating SpMM with heterogeneous accelerator architectures," in *Proc. HPCA*, May 2024, pp. 1012–1028.
- [31] M. Soltanyeh, R. P. Martin, and S. Nagarakatte, "An accelerator for sparse convolutional neural networks leveraging systolic general matrix-matrix multiplication," *ACM Trans. Archit. Code Optim.*, vol. 19, no. 3, pp. 1–26, Sep. 2022.
- [32] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2019, pp. 151–165.
- [33] J. Yang et al., "BP-im2col: Implicit im2col supporting AI backpropagation on systolic arrays," in *Proc. IEEE 40th Int. Conf. Comput. Design (ICCD)*, Sep. 2022, pp. 415–418.
- [34] Y. Cao et al., "MZ core: An enhanced matrix acceleration engine for HPC/ AI applications," in *Proc. IEEE 24th Int. Conf. High Perform. Comput. Commun.; 8th Int. Conf. Data Sci. Syst.; 20th Int. Conf. Smart City; 8th Int. Conf. Dependability Sensor, Cloud Big Data Syst. Appl. (HPCC/DSS/SmartCity/DependSys)*, Dec. 2022, pp. 126–134.
- [35] F. Li, G. Li, Z. Mo, X. He, and J. Cheng, "FSA: A fine-grained systolic accelerator for sparse CNNs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 11, pp. 3589–3600, Nov. 2020.
- [36] M. Mahmoud et al., "TensorDash: Exploiting sparsity to accelerate deep neural network training," in *Proc. MICRO*, Oct. 2020, pp. 781–795.
- [37] L. Lu et al., "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *Proc. MICRO-54: 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2021, pp. 977–991.
- [38] X. Chen, J. Zhu, J. Jiang, and C.-Y. Tsui, "Tight compression: Compressing CNN through fine-grained pruning and weight permutation for efficient implementation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 644–657, Feb. 2023.
- [39] B. Asgari, R. Hadidi, and H. Kim, "MEISSA: Multiplying matrices efficiently in a scalable systolic architecture," in *Proc. IEEE 38th Int. Conf. Comput. Design (ICCD)*, Oct. 2020, pp. 130–137.



Minjin Tang received the B.S., M.S., and Ph.D. degrees from the National University of Defense Technology, Changsha, China, in 2018, 2020, and 2025, respectively.

He currently works with the Defense Innovation Institute, Academy of Military Science, Beijing, China. His research interests include computer architecture and sparse processing.



Mei Wen received the B.S., M.S., and Ph.D. degrees in computer science and technology from the National University of Defense Technology, Changsha, China, in 1995, 1999, and 2006, respectively.

She is currently a Professor with the Computer College, National University of Defense Technology. Her research interests include computer architecture, parallel programming, and scientific computing.